

期末專題報告

AWS CI/CD 演化闖關

從 18 個步驟、16 分鐘的人工部署，
到 1 個步驟、80 秒的全自動流程

題目：P3 CI/CD 演化闖關 (Tier 3)

完成關卡：關 0 ～ 關 4

姓名：張博堯

日期：2026 年 9 月

程式碼與完整證據：github.com/CarlosChangYao/aws-cicd-pipeline

目錄

(在 Word 中按 F9 或右鍵「更新功能變數」可產生頁碼)

摘要

本專題以「工程師改了一行程式碼，這行程式碼要怎麼變成線上正在跑的網站」為題，將部署流程從最原始的人工作業，逐關演化為完全自動化的持續整合 / 持續部署流水線，並在每一關記錄可量測的數據。

與多數以「學會某項工具」為目標的做法不同，本專題刻意先完成一次完整的手動部署，記錄其步驟數、耗時與失敗次數，作為後續各關的對照基準。此基準線使每一關的價值可被量化，而非僅止於宣稱。

主要成果：

項目	關 0（手動）	關 3（全自動）	改善
人工步驟數	18 步	1 步	-94%
端到端耗時	16 分鐘	80 秒	-92%
人工介入	每次都要	零	—
失敗次數	2 次	0 次	—
自動化測試關卡	無	有，不過則中止	—
版本可追溯	否	精確對應 commit	—
換版服務中斷	有	無（關 4，實測 100% 可用）	—

整條流水線在設計上不存放任何長期憑證：GitHub Actions 以 OIDC 換取一小時有效的臨時憑證，主機以 IAM 角色取得權杖，部署經由 Systems Manager 執行而不使用 SSH。基礎設施全部以 CloudFormation 管理。

一、專題背景與選題

1.1 題目與範圍

本專題選擇題號 P3「CI/CD 演化闖關」，屬 Tier 3。該題設計為六關遞進，規則為「停在哪關，拿哪關的分」。

關卡	內容	本專題狀態
關 0	手動部署到 EC2（體會痛點）	完成
關 1	Dockerfile → 推送 ECR → EC2 拉取	完成
關 2	push GitHub → Actions 自動建置 → 進 ECR	完成
關 3	ECR 更新 → 自動部署到 EC2	完成（Tier 3 成立）
關 4	改自動部署到 ECS Fargate	完成
關 5	跨雲：自動分發至 GCE 與 Cloud Run	評估後不執行

1.2 選題理由

選題時的主要考量有三：

- 風險可控：六關遞進且「停在哪關拿哪關的分」，任何一週落後，先前關卡的成果仍是完整可交付的。
- 延用既有能力：Docker（課程第 10-11 週）、Git（第 3 週）、Linux（第 5-9 週）、VPC 與 IAM（雲端架構課程）皆為課程既有內容，真正的新學習項目僅 ECR、GitHub Actions 與 OIDC 三項。
- 實務關聯性：CI/CD 是現代開發團隊的日常標配，且本題的核心產出「六關前後對照」可直接說明自動化的商業價值，而非僅止於技術展示。

1.3 展示用應用程式的設計

本專題刻意選用極簡的 Flask 應用（約 45 行），首頁僅顯示版本號、Git commit 編號、部署時間與服務主機名稱。

理由是：本題的主角是「流水線」而非「程式功能」。應用程式越簡單，越能將注意力集中於部署自動化本身。而將版本資訊做成畫面上可見的內容，是為了讓驗收當下「改一行程式碼、重整瀏覽器、數字真的變了」能被直接觀察到，不需檢視日誌。

二、系統架構

2.1 整體資料流

- 1. 開發者於本機修改程式碼，執行 git push（唯一的人工動作）
- 2. GitHub 偵測到推送事件，觸發 GitHub Actions 流水線
- 3. 階段一：執行 pytest 測試。未通過則流水線中止，不進行後續任何步驟
- 4. 階段二：以 OIDC 換取 AWS 臨時憑證，建置容器映像檔並推送至 Amazon ECR
- 5. 階段三：透過 Systems Manager Run Command 指示 EC2 拉取新映像檔並重啟容器
- 6. 階段四：更新 ECS 任務定義，觸發 Fargate 滾動更新（與階段三平行執行）
- 7. 部署完成後自公開網址進行健康檢查，確認線上版本與本次建置一致

2.2 網路架構

元件	設定	設計理由
VPC	10.0.0.0/16	自建而非使用 default VPC，才能自行控制網段切分
公有子網 ×2	10.0.1.0/24 (1a) 10.0.2.0/24 (1c)	放置對外服務；ALB 規定至少跨兩個可用區
私有子網 ×2	10.0.11.0/24 (1a) 10.0.12.0/24 (1c)	使用獨立路由表且無對外路由
Internet Gateway	掛載於 VPC	公有子網的對外通道
NAT Gateway	刻意不建置	私有子網目前無主動對外需求。省下約 US\$32/月，並減少一個攻擊面
Security Group	應用主機僅開放 80	不開放 22；管理一律經由 Session Manager

關於私有子網的說明：許多實作以 Security Group 阻擋私有資源對外，本專題採取的做法是讓私有子網使用獨立的路由表，且該路由表僅含本地路由、不含任何指向 Internet Gateway 的條目。

兩者的差異在於：Security Group 是防火牆，管制的是「誰可以連進來」；路由表決定的是「有沒有路可以出去」。沒有路由，封包在網路層即無法離開，比倚賴防火牆規則更為徹底。

2.3 運算與部署目標

	關 3：EC2	關 4：ECS Fargate
運算形式	單一 EC2 執行 Docker 容器	Fargate 任務，無主機需管理
數量與分佈	1 台，位於可用區 1a	2 個任務，分佈於 1a 與 1c
流量入口	Elastic IP 直連	Application Load Balancer
健康檢查	容器層 HEALTHCHECK	ALB 主動探測 /healthz
換版方式	先終止舊容器再啟動新容器	滾動更新：新任務健康後才終止舊任務

	關 3： EC2	關 4： ECS Fargate
換版中斷	有（數秒）	無

兩個部署目標並存為刻意設計，目的在於驗收時可同時展示關 3 與關 4 的差異，使「滾動更新」的價值有直接對照。

三、實作歷程

3.1 關 0：手動部署——建立基準線

本關的目標不是「部署成功」，而是「記錄部署有多痛」。手動部署必然會成功，其價值在於產出後續各關的對照基準。

實作方式：以 Session Manager 連入 EC2（未開放 22 port、無 SSH 金鑰），人工建立目錄、貼入程式碼、建立 Python 虛擬環境、安裝套件、人工填寫版本資訊、撰寫 systemd 服務檔並啟動。

量測項目	數值	依據
開始時間	11:40:12	截圖 09（Session Manager 連線成功）的檔案時間戳
部署完成	11:56:15	截圖 10（瀏覽器驗證成功）的檔案時間戳
總耗時	約 16 分鐘	兩張截圖的時間差，非事後估算
總指令數	18 步	含 3 步因失敗而重做
失敗次數	2 次	詳見第六章
版本可追溯	否	GIT_COMMIT 欄位只能填 none-manual-deploy

本關記錄的七項痛點（後續各關逐一解決）：

- 無檔案傳輸機制，程式碼只能整段貼入終端機，內容較長時必然出錯
- 版本號為人工填寫，無任何機制保證其與實際執行的程式碼一致
- GIT_COMMIT 只能填寫佔位字串，代表無法追溯、無法回滾
- 每修改一行程式碼，上述 18 個步驟需從頭執行一次
- 無測試關卡，程式是否損壞僅能倚賴操作者記得手動檢查
- 環境依賴需人工處理，換一台機器即需重做且無法保證環境一致
- 全程仰賴操作者不出錯，18 個步驟中無任何一步具備防呆

3.2 關 1：容器化與 ECR

將程式與其執行環境一併打包為容器映像檔，推送至 Amazon ECR 私有倉庫，由 EC2 拉取執行。

關 0 的問題	關 1 的解法	驗證結果
程式碼手貼會壞（失敗 2 次）	docker pull 完整傳輸	傳輸零失敗
要人工建虛擬環境、裝套件	環境打包進映像檔	EC2 上未安裝任何套件
換台機器就要全部重來	同一映像檔行為一致	Mac 建置、EC2 執行
不知道線上是哪一版	版本資訊烤進映像檔	GIT_COMMIT = 1cfbef1

計量結果：部署步驟由 18 步降至 7 步；機器執行總時間約 37 秒（建置 17.7 秒、推送 10.6 秒、拉取 3.5 秒、啟動約 5 秒）；映像檔大小 47.4 MB；傳輸失敗 0 次。

本關首次使「線上執行的是哪一版程式碼」成為可查證的事實——版本資訊由 `git rev-parse` 取得後烤進映像檔，可回溯至 GitHub 上該次修改的完整內容。

3.3 關 2：GitHub Actions 自動建置

將關 1 仍需人工執行的建置與推送自動化，並加入測試關卡。

流水線設計為兩段式 job，後段以 `needs` 相依於前段：

- job 1「執行測試」：安裝相依套件並執行 `pytest`
- job 2「建置並推送」：僅在 job 1 成功時執行；以 OIDC 取得憑證、建置映像檔、推送 ECR

此設計在技術上稱為持續整合，若以金融業的語言表述，則是「自動化的內控關卡」——過往變更管理倚賴人工簽核與人工記得檢查，此處改為機器強制執行，無法繞過。

版本號的產生方式亦有改變。關 0 與關 1 的版本號由人工決定，關 2 之後改由 GitHub 的流水線執行序號與 commit SHA 自動產生，人工無從介入，因此不可能填錯。

推送時同時打上三個標籤，各有用途：

標籤	用途
<commit-sha>	精確對應某次程式碼修改，回滾時指定此標籤
<version>	人類可讀的版本號
latest	一般部署預設拉取

計量結果：人工步驟 1 步（`git push`）；流水線執行 56 秒；人工介入零。

3.4 關 3：自動部署至 EC2

關 2 完成後，映像檔已自動進入 ECR，但 EC2 仍執行舊版本——因為尚無任何機制通知它。本關補上此斷點。

實作方式：GitHub Actions 呼叫 Systems Manager 的 `SendCommand`，指示 EC2 拉取指定映像檔並重啟容器。選擇此方式而非 SSH 的理由有三：

- EC2 不需開放任何 inbound port
- GitHub 不需保管任何主機金鑰

- 每次部署皆留下 CloudTrail 稽核軌跡，且綁定到具體身分

端到端驗證（檢核點 #17）：

時間	事件
14:22:49	執行 git push（唯一的人工動作）
—	自動觸發：測試 → 建置 → 推送 ECR → SSM 部署
14:24:09	線上版本由 v1.2.8-ci 變更為 v1.2.9-ci
合計 80 秒	期間未執行任何人工動作

可追溯性形成閉環：線上網頁顯示的 Git Commit 為 dc4758f，與本機 git log 的最新一筆完全一致。相較於關 0 該欄位只能填寫 none-manual-deploy，此處「線上執行的是哪一版」已成為百分之百確定的事實。

另一項設計決策為：部署時指定 commit SHA 標籤而非 latest。latest 會被後續建置覆蓋，唯有 SHA 能保證「部署的版本」與「程式碼」永久對應，回滾時亦才有明確依據。

3.5 關 4：ECS Fargate 滾動更新

關 3 的部署動作為先終止舊容器、再啟動新容器，兩者之間必然存在數秒空窗。單機部署無法避免此問題——僅有一個容器，終止後即無服務。

在金融業，此問題對應到「上線需要停機窗口」：必須排在收盤後或週末執行，並動員人力待命。

關 4 的解法為滾動更新，其核心為兩項部署參數：

```
MinimumHealthyPercent: 100    # 部署期間健康任務數不得低於原本的 100%
MaximumPercent: 200           # 允許暫時擴充至 200%
```

兩者合併的效果為：先啟動新任務，等待其通過 ALB 健康檢查，才逐一終止舊任務。全程至少維持原有數量的健康任務。並搭配 Target Group 的除役延遲設定，使舊任務先停止接收新連線、待既有連線處理完畢後才終止。

3.5.1 零停機驗證

方法：於部署期間以每 0.5 秒一次的頻率持續請求 ALB，記錄每次的 HTTP 狀態碼與回傳版本。

項目	數值
總請求數	300 次
HTTP 200	300 次
失敗（非 200 或連線失敗）	0 次

項目	數值
可用率	100.00%

換版瞬間的實際紀錄：

15:11:26	200	v1.2.13-ci	(舊版)
15:11:27	200	v1.2.14-ci	(新任務通過健康檢查，開始分流)
15:11:28	200	v1.2.13-ci	(舊任務仍在服務)
15:11:31	200	v1.2.14-ci	
15:11:32	200	v1.2.13-ci	
： 兩版並存約 15 秒，流量逐步轉移			
15:11:47	200	v1.2.14-ci	(舊任務全數下線，切換完成)

新舊版本同時服務約 15 秒，期間無任何請求失敗。此交錯模式為滾動更新的直接證據——它顯示系統為「先啟動新的、確認健康、再收回舊的」，而非「先終止再啟動」。

四、成果與量化分析

4.1 五關對照表

	人工步驟	端到端耗時	人工介入	測試關卡	版本可追溯	換版中斷
關 0 手動部署	18 步	16 分鐘	每次都要	無	否	有
關 1 容器化	7 步	37 秒	每次都要	無	手動帶入	有
關 2 自動建置	1 步	56 秒	零	自動阻擋	機器產生	有
關 3 自動部署	1 步	80 秒	零	自動阻擋	對應 commit	有
關 4 滾動更新	1 步	約 2 分鐘	零	自動阻擋	對應 commit	無

關 0 至關 3 的改善幅度：人工步驟減少 94%、端到端耗時減少 92%、人工介入歸零、新增自動化測試關卡、版本從無法追溯轉為精確對應 commit。

4.2 關於耗時數據的說明

表中關 1 的 37 秒短於關 3 的 80 秒，此處需說明其定義差異，以免誤讀：

- 關 1 的 37 秒為「機器執行的時間」。該關之後，操作者仍須自行打指令建置、自行推送、自行部署。
- 關 3 的 80 秒為「自按下 git push 起，至使用者可看見新版本止」的完整時間，期間無任何人工動作。
- 關 4 的耗時再增加，是因為滾動更新必須等待新任務通過健康檢查後才能收回舊任務。此段時間是為換取「零中斷」而刻意付出的成本，並非效能退步。

五、安全設計

5.1 整條流水線零長期憑證

本專題最主要的安全設計為：整條流水線中不存放任何長期金鑰。

環節	認證方式	憑證有效期
GitHub Actions → AWS	OIDC 身分聯邦換取臨時憑證	1 小時
Actions → ECR	以上述臨時憑證換取推送權杖	12 小時
Actions → EC2	SSM Run Command（不經 SSH）	單次
EC2 → ECR	EC2 IAM Role 換取權杖	12 小時

常見的做法是產生一組 AWS access key 並存入 GitHub Secrets，此舉等同將帳號的操作權限永久交付第三方平台保管。本專題改採 OIDC：GitHub 於每次流水線執行時簽發一張數分鐘有效的身分權杖，AWS 驗證其確實來自指定 repo 後，才發給一小時後即失效的臨時憑證。結果是 GitHub 端與 AWS 端皆無金鑰存在。

5.2 其他安全措施

措施	實作方式	意義
不開放 SSH	SG 不放行 22，且主機內 sshd 已停用	縱深防禦：即使 SG 遭誤改，主機亦無 SSH 服務可連
容器非 root 執行	建立專用使用者並切換	容器遭入侵時攻擊者非 root
映像檔漏洞掃描	ECR scanOnPush	實際掃出 CRITICAL 4、HIGH 8、MEDIUM 6（來源為基礎映像檔）
IAM 最小權限	權限精確至單一資源	只能推特定倉庫、只能對特定主機下指令、只能更新特定服務
私有子網無對外路由	獨立路由表僅含本地路由	由路由層保證，非倚賴防火牆規則
基礎設施即程式碼	CloudFormation	整組建立、整組刪除，避免資源殘留
成本控制	Budgets 告警、ECR 生命週期政策	超支前告警；映像檔僅保留最近 5 版

5.3 權限的漸進式授予

GitHub Actions 所使用的 IAM 角色，其權限並非一次給足，而是隨各關的實際需求逐步追加：

政策名稱	加入時機	授權範圍
ecr-push-nkc202-cicd-app	關 2	僅能推送至 nkc202-cicd-app 這一個 ECR 倉庫

政策名稱	加入時機	授權範圍
ssm-deploy-single-instance	關 3	僅能對 i-0937ad0e32c5fedca 執行 AWS-RunShellScript
ecs-deploy-single-service	關 4	僅能更新 nkc202-cicd-service 這一個服務

此做法體現最小權限原則的實務意涵：需要時才給，不預先授予。

六、問題排除紀錄

6.1 GitHub OIDC 信任政策比對失敗（關 2）

本專題耗時最久、也最具啟發性的問題。自首次執行至成功共歷經 6 次流水線執行，錯誤訊息始終為同一句：

```
Error: Could not assume role with OIDC:
Not authorized to perform sts:AssumeRoleWithWebIdentity
```

該訊息僅表明「不被允許」，未指出何項條件不符，資訊量極低。排除過程如下：

執行次數	採取的行動	結果
第 1 次	首次執行	失敗
第 2 次	推測①：IAM 設定傳播延遲，重新觸發	推測錯誤
第 3 次	加入除錯步驟，印出 OIDC 權杖的實際內容	仍失敗，但取得關鍵資料
第 4 次	推測②：憑證指紋過期，更新為即時抓取的指紋	推測錯誤
第 5 次	推測③：大小寫不符，信任政策同時接受兩種寫法	推測錯誤
第 6 次	讀取除錯輸出後對症下藥	成功

真正的原因：

除錯步驟印出 GitHub 實際送出的身分內容為：

```
sub = repo:CarlosChangYao@39613248/aws-cicd-pipeline@1334878950:ref:refs/heads/main
```

而信任政策設定的是網路教學文件上常見的舊格式：

```
repo:CarlosChangYao/aws-cicd-pipeline:*
```

GitHub 已改用「不可變主體識別」格式，於 sub 欄位中嵌入帳號與 repo 的數字識別碼。兩種格式並不相同，因此比對必然失敗。修正後即通過。

附帶說明：改用數字識別碼實際上更為安全——帳號名稱與 repo 名稱皆可變更，數字識別碼則不會。即使日後有人註冊同名帳號與 repo，亦無法通過驗證。

本次經驗的啟示：

三次推測全數錯誤，共耗費約 10 分鐘；加入除錯步驟印出實際資料後，問題於數秒內定位完成。當錯誤訊息的資訊量不足時，唯一有效的方法是將雙方實際的值印出直接比對，而非依經驗猜測。此工作習慣亦貫穿本專題其餘各關。

6.2 跨架構建置（關 1）

開發機為 Apple Silicon (arm64)，目標 EC2 為 x86_64。映像檔內含已編譯的機器碼，兩種架構不相容，直接建置的映像檔於 EC2 上執行會出現 exec format error。

解法為建置時指定 --platform linux/amd64。附帶效益是關 2 之後改由 GitHub Actions 建置，其執行環境本即為 x86_64，不再需要跨架構模擬。

6.3 ECR 漏洞掃描無結果（關 1）

倉庫已設定 scanOnPush，但查詢掃描結果回報 ScanNotFoundException。

根本原因為 buildx 搭配 --platform 參數時，預設會產出 OCI Image Index（映像檔索引，內含架構清單與 attestation 中繼資料），而 ECR 基本掃描僅支援單一映像檔 manifest，無法處理索引。

解法為建置時加上 --provenance=false --sbom=false，產出單一架構 manifest 後掃描即正常運作，並實際掃出 CRITICAL 4、HIGH 8、MEDIUM 6 項漏洞。

6.4 CloudFormation 範本欄位編碼限制（關 0）

首次建立基礎環境時，Security Group 建立失敗並回滾。錯誤訊息為 Character sets beyond ASCII are not supported，原因是 GroupDescription 欄位填入了中文。

該欄位會被轉送至 EC2 API，而該 API 僅支援 ASCII。範本中其他中文（如 Tag 的值、Parameters 與 Outputs 的說明）則由 CloudFormation 自行處理，不受此限制。

此次失敗亦驗證了 CloudFormation 的自動回滾機制：建置失敗時，已建立的 11 個資源全數自動刪除，帳號中未留下任何孤兒資源。若以 CLI 逐條建立，失敗時需自行尋找並清除。

七、設計決策與取捨

7.1 主要決策一覽

決策	選擇	理由	代價
NAT Gateway	不建置	私有子網無主動對外需求	省 US\$32/月，並少一個攻擊面
私有子網對外	獨立路由表、無對外路由	由路由層保證，非倚賴防火牆	無
主機管理方式	SSM，並停用 sshd	縱深防禦，不倚賴單一控制點	無
Actions 認證	OIDC 而非 Access Key	repo 內零長期金鑰	設定較複雜（卡了 6 次執行）
部署指定標籤	commit SHA 而非 latest	確保部署版本與程式碼精準對應	無
容器執行身分	非 root	遭入侵時攻擊者非 root	需將監聽埠改為 8080 再對外映射
Fargate 網路位置	公有子網並配發公有 IP	任務需拉取 ECR。NAT 約 US\$32/月、VPC Endpoints 約 US\$21/月，本方案 US\$0；安全性由 SG 保證僅接受 ALB 流量	正式環境仍建議私有子網加 VPC Endpoints
ECR 保留版本數	5 版	避免映像檔無限累積產生儲存費	更舊的版本無法回滾
基礎設施管理	CloudFormation	整組建立與刪除，避免資源殘留	學習成本

7.2 關 5 的評估與決定

關 5 為跨雲分發，需將映像檔同時部署至 Google Cloud 的 GCE 與 Cloud Run。本專題評估後決定不執行。

評估項目	內容
需完成的工作	另建 GCP 帳號並綁定帳單、設定 Workload Identity Federation、學習 Cloud Run 部署模型、撰寫並維護第二套流水線
預估投入	5 至 8 小時
對 Tier 認定的影響	無，仍為 Tier 3
替代用途	同樣時間投入關 4 的滾動更新與文件完整度
決定	不執行

事後檢視，此判斷成立：投入關 4 所產出的「300 次請求零失敗」數據，是本報告最有力的證據之一。若將該時間投入跨雲，將多一個關卡，卻少一項可量化的成果。

「評估過、有理由地決定不做」與「沒做完」是兩件不同的事。架構設計的工作不僅是將系統做出來，亦包含判斷何者不該做，並說明其依據與代價。

八、與金融業實務的對應

本專題的技術成果之外，其核心論述為：在金融業，「系統上線」並非純技術問題，而是風險與法遵問題。

一般科技公司上線出錯，使用者反映後修復即可；證券公司上線出錯，客戶可能無法下單、可能見諸新聞、可能面臨主管機關檢查。因此金融業對上線的態度為「寧可慢，不可錯」，流程遂演變為：排定上線窗口、收盤後或週末執行、資訊與業務單位共同待命、人工執行、出錯時人工回復。

然而這套以安全為名的做法，本身即製造了數項風險。

手動部署的風險	對應的內控概念	本專題的解法
18 個步驟仰賴人不出錯	作業風險	自動化，人工僅需 1 步
不知道線上執行的是哪一版	稽核軌跡 / 可追溯性	每次部署綁定 commit SHA
出錯後復原耗時	營運持續 (BCP)	指定舊版 SHA 重新部署即可回滾
上線需人工登入主機	存取控制 / 特權帳號管理	全程無 SSH，操作留下 CloudTrail 紀錄
缺乏審批與檢查紀錄	變更管理 / 內控三道防線	測試未通過流水線即中止，無法繞過
換版需停機窗口	服務中斷 / 服務水準	滾動更新，營業時間內即可上線
不知曉第三方元件的漏洞	軟體供應鏈安全	ECR 推送時自動掃描並回報

其中「軟體供應鏈安全」一項值得補充。2021 年 Log4j 事件期間，多數金融機構面臨的最大困難並非修補本身，而是「無法確知哪些系統使用了該元件」。本專題所實作的映像檔推送掃描，即為此類風險的主動盤點機制。

此外，金管會已鬆綁金融機構委外上雲之規範，取消跨境委外一律須申請核准的要求並簡化流程。可預期此類需求在證券業將持續增加。

九、結論與後續改善

9.1 結論

本專題完成關 0 至關 4，Tier 3 達標。將部署流程自 18 個步驟、16 分鐘的人工作業，改造為 1 個步驟、80 秒的全自動流程，並於關 4 達成換版零中斷（實測 300 次請求全數成功）。

回顧整個歷程，本專題真正的產出並非五個完成的關卡，而是兩份紀錄：

其一，關 0 的基準線。

18 步、16 分鐘、2 次失敗、版本無法追溯。若無此四項數據，後續各關僅是四項工具的學習成果；有了這些數據，它們才成為四個具體問題的解決方案，且改善幅度可被量化。

其二，關 2 的六次失敗歷程。

三次推測全數錯誤、一次量測直接命中。本報告刻意完整保留三次錯誤推測的過程，因為它所證明的並非「能夠使用工具」——照著教學文件操作亦能使用工具——而是「在沒有現成答案時，能夠找出問題所在」。

9.2 後續改善方向

項目	現況	改善方向
基礎映像檔漏洞	掃出 CRITICAL 4、HIGH 8	定期更新基底映像檔，並將掃描結果納入流水線的檢查條件，超過門檻即中止
Fargate 網路位置	公有子網並配發公有 IP	改用私有子網加 VPC Endpoints，代價為每月約 US\$21
監控與告警	僅有健康檢查與日誌	加入 CloudWatch 告警與異常通知，並建立服務水準指標
HTTPS	僅提供 HTTP	申請憑證並於 ALB 終止 TLS
回滾自動化	需人工指定舊版 SHA	健康檢查連續失敗時自動回滾至前一版
跨可用區資料層	本專題無資料層	若導入資料庫，須考量多可用區部署與備份策略

附錄 A：檢核點對照

#	檢核項目	狀態	證據
1	應用可執行，首頁顯示版本號	完成	關 0 截圖 10、11
2	外部瀏覽器連得到（手機行動網路）	完成	關 0 截圖 11（畫面顯示 4G）
3	EC2 位於正確的 VPC 與子網	完成	關 0 截圖 07
4	記錄手動部署的步驟數與耗時	完成	關 0 紀錄檔（18 步 / 16 分鐘）
5-9	Dockerfile、ECR 倉庫、雙標籤推送、EC2 拉取、IAM Role 存取	完成	關 1 截圖 01-03
10-11	GitHub repo、workflow 自動觸發	完成	關 2 截圖 01-04
12	測試失敗時流水線中止	待補	機制已建置（needs 相依），待實際演示
13-14	自動 build 推 ECR、OIDC 零金鑰	完成	關 2 截圖 05-06
15-17	SSM 觸發部署、容器自動更新、端到端驗證	完成	關 3 截圖 01-04
18-19	全程無人工登入、不開 22 port	完成	關 0 截圖 08、外部連接埠實測
20	指定舊版本回滾	待補	機制已建置（SHA 標籤），待實際演示
21-23	ECS 建置、Actions 更新服務、滾動不中斷	完成	關 4 紀錄檔（300 次請求零失敗）
24-27	架構圖、各關證據截圖、VPC 截圖、README	完成	docs/架構圖.md、26 張截圖、README.md
28	六關前後對照表	完成	本報告第四章
29	開帳先設 Budgets 告警	完成	關 0 證據
30	Demo 後資源全數清除	待辦	驗收後執行

附錄 B：使用的 AWS 服務與資源

類別	服務 / 資源
運算	EC2（t3.micro）、ECS Fargate（0.25 vCPU / 0.5 GB × 2）
網路	VPC、Subnet × 4、Internet Gateway、Route Table × 2、Security Group × 4、Application Load Balancer、Elastic IP
容器	Amazon ECR（私有倉庫，啟用推送掃描與生命週期政策）

類別	服務 / 資源
身分與存取	IAM Role × 5、IAM User、OIDC Identity Provider
管理	Systems Manager（Session Manager、Run Command）
可觀測性	CloudWatch Logs、ALB 健康檢查、容器 HEALTHCHECK
基礎設施即程式碼	CloudFormation（兩份範本）
成本控制	AWS Budgets
外部服務	GitHub、GitHub Actions
應用技術	Python 3.11、Flask、gunicorn、pytest、Docker

附錄 c：交付物索引

全部內容公開於 github.com/CarlosChangYao/aws-cicd-pipeline

檔案 / 目錄	內容
README.md	專案總覽、成果對照、安全設計、除錯歷程
docs/架構圖.md	完整架構圖、六關演進圖、認證流程時序圖、設計決策表
app/	Flask 應用、測試、Dockerfile
infra/01-base.yaml	VPC、子網、Security Group、IAM、EC2
infra/02-ecs.yaml	ALB、ECS Cluster、Fargate Service
.github/workflows/	四段式流水線定義
關 0～關 4 紀錄檔（5 份）	各關的目標、做法、計量、問題排除、設計決策
證據截圖/（26 張）	關 0 13 張、關 1 3 張、關 2 6 張、關 3 4 張
docs/關 4_零停機量測原始資料.log	300 筆請求的原始紀錄